



MCDI500 • PROGRAMACIÓN PARA LA CIENCIA DE DATOS
MAGÍSTER EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL
UNIVERSIDAD ANDRÉS BELLO

Análisis de ofertas en licitaciones públicas del sector Salud

Proyecto final integrador Fase 4: análisis, reproducibilidad y comunicación de resultados

Repositorio: github.com/vbravo29/mcdi500-licitaciones-salud

Grupo 5

Víctor Bravo Barrera

Nayadeth Garrido Ibáñez

Mauricio Cid

Docente: Omar Salinas Silva

Fecha: 4 de octubre de 2026

Índice de contenidos

1. Introducción y contextualización	3
2. Problema, objetivos e hipótesis	3
3. Metodología del desarrollo técnico	3
3.1 Integración de las fases F1-F4	3
3.2 Herramientas, reproducibilidad y control de versiones	4
3.3 Eficiencia, validación y programación orientada a objetos	5
4. Resultados y discusión	6
4.1 Resultados por tamaño de proveedor	6
4.2 Resultados por tipo de licitación	7
4.3 Plazos de cierre y discusión	8
5. Conclusiones y recomendaciones	8
6. Trazabilidad de mejoras	9
7. Bibliografía	10

1. Introducción y contextualización

Este informe integra las cuatro fases del proyecto de análisis de ofertas en licitaciones públicas del sector Salud, de acuerdo con los lineamientos de la Fase 4 del curso (Universidad Andrés Bello, 2026a, 2026b). El trabajo utiliza el reporte de Datos Abiertos de ChileCompra de marzo de 2026, compuesto por 44.226 ofertas y 74 variables (ChileCompra, s. f.). La unidad de observación es una oferta asociada a un ítem; por ello, una misma licitación puede aparecer varias veces. El análisis describe diferencias en la proporción de ofertas ganadoras y en los plazos de cierre, con un flujo reproducible que conserva el archivo original, valida cada transformación y comunica los resultados mediante tres visualizaciones.

El problema es relevante porque los porcentajes pueden inducir conclusiones erróneas si se mezclan procesos no adjudicados, se usa un denominador global o se ignora que ciertas categorías tienen muy pocos casos. Resolverlo requiere programación para leer y transformar el archivo de forma consistente, pruebas para detectar errores, mediciones para escoger una implementación eficiente y visualizaciones que permitan interpretar los patrones sin confundir una descripción con una relación de causa y efecto. Cada fase aporta una parte de esa solución: F1 define el problema y el entorno, F2 prepara los datos, F3 construye el núcleo algorítmico y F4 comunica los resultados.

2. Problema, objetivos e hipótesis

Problema. Se requiere establecer cómo varía la proporción de ofertas ganadoras según el tamaño del proveedor y el tipo de licitación, y si los procesos de distinto monto ofrecen plazos de cierre diferentes. El cálculo se restringe a procesos adjudicados y usa como denominador las ofertas con resultado válido de cada grupo.

Objetivo general. Desarrollar y documentar un flujo reproducible en Python que prepare, valide, analice y comunique los patrones de adjudicación del sector Salud observados en marzo de 2026.

Objetivos específicos. (1) Conservar la trazabilidad entre datos originales y resultados; (2) comparar alternativas algorítmicas equivalentes; (3) aplicar una arquitectura modular y orientada a objetos; (4) contrastar la hipótesis mediante tres visualizaciones; y (5) documentar límites, decisiones y mejoras entre F1 y F4.

Hipótesis. La proporción de ofertas ganadoras aumenta con el tamaño del proveedor: las empresas grandes ganan una mayor parte de sus ofertas que las micro y pequeñas. Las preguntas complementarias examinan si esa proporción cambia según el tipo de licitación y si el plazo de cierre aumenta con el monto del proceso.

Alcance y supuestos. El análisis es descriptivo, cubre un mes y un sector, y no demuestra causalidad. Se supone que las etiquetas de resultado y tipo de licitación reflejan el registro administrativo; las fechas del año 1900, usadas como valor de relleno en el registro, no se interpretan como fechas reales; y la categoría NoClasificado se conserva para no asignar un tamaño empresarial sin evidencia.

3. Metodología del desarrollo técnico

3.1 Integración de las fases F1-F4

El proyecto se construyó como una secuencia acumulativa. Cada fase dejó un notebook ejecutable y un registro de su ejecución, de modo que cada mejora pudiera compararse con la versión anterior (Tabla 1).

Tabla 1*Síntesis metodológica de las fases del proyecto*

Fase	Trabajo realizado	Evidencia principal
F1	Definición del problema, unidad de análisis, fuente y entorno reproducible.	F1_Definición.ipynb
F2	Exploración de 74 variables; limpieza, fechas, categorías, variables derivadas y validación.	F2_Preprocesamiento.ipynb
F3	Separación modular, POO, cuatro algoritmos equivalentes, pruebas y mediciones.	F3_Algoritmos.ipynb, 29 pruebas
F4	Reutilización del flujo validado, tres gráficos, interpretación y comunicación final.	F4_Comunicacion.ipynb, 10 pruebas

Nota. Elaboración propia a partir de los notebooks y verificadores del repositorio.

En F2 se eliminaron únicamente tres columnas completamente vacías; 7.613 valores del año 1900 en la fecha estimada de evaluación se convirtieron en valores ausentes; se normalizaron espacios en categorías; y se calcularon oferta_ganadora, licitacion_adjudicada y plazo_cierre_dias, siguiendo las prácticas de preparación de datos con pandas descritas por McKinney (2022). No se imputaron los faltantes parciales, porque las variables del análisis estaban completas, ni se escalaron variables, porque no se entrenó un modelo que lo requiriera. Las 44.226 filas se conservaron.

El flujo final es: CSV original → verificación SHA-256 → lectura de siete columnas → limpieza sin modificar los datos de entrada → seis reglas de calidad → cálculo agrupado → tablas y figuras. El siguiente fragmento muestra cómo se encadenan esos pasos en los notebooks de F3 y F4:

```
lector = LectorCSV(contrato, columnas=COLUMNAS_ANALISIS, tipos=TIPOS_ANALISIS)
raw = lector.leer(ruta) # valida el contrato de columnas
datos = LimpiadorLicitaciones().limpiar(raw) # no modifica la entrada
ValidadorDatasetProcesado().validar(datos, len(raw)) # seis reglas de calidad
tabla = tabla_proporciones_agrupada(datos, "TamanoProveedor")
pd.testing.assert_frame_equal(tabla, tabla_proporciones(datos, "TamanoProveedor"))
```

La última línea comprueba que las proporciones coincidan con la función de referencia de F2. Así se asegura que ninguna optimización cambie los resultados del análisis.

3.2 Herramientas, reproducibilidad y control de versiones

El proyecto combina Python, NumPy, pandas, Jupyter, unittest, Matplotlib (Hunter, 2007) y Seaborn (Waskom, 2021) para el análisis, y Git con GitHub para el control de versiones y la colaboración. El archivo requirements.txt fija NumPy 2.3.5 y pandas 3.0.1 y declara el resto de las dependencias; el entorno se crea con venv. Los verificadores ejecutan cada notebook en un kernel nuevo y guardan en evidencias/ la fecha, la versión de Python, el número de celdas, los errores y la huella del notebook, en línea con las recomendaciones de reproducibilidad de Samuel y Mietchen (2024). F1 y F2 se verificaron con Python 3.12.14, F3 con 3.11.15 y F4 con 3.14.7, sin cambios en los resultados. El CSV original permanece versionado y los derivados reproducibles quedan fuera de Git para no duplicar archivos grandes.

El control de versiones se gestionó con Git y el repositorio se aloja en GitHub (github.com/vbravo29/mcdi500-licitaciones-salud). El repositorio tiene una carpeta por fase (F1 a F4), el código común en src/ y un README con instrucciones de instalación y ejecución. El trabajo se

integra en la rama desarrollo y pasa a main mediante pull request en GitHub, lo que deja un registro revisable de cada integración. El archivo docs/changelog.md vincula cada mejora con su commit. El historial supera los 60 commits y registra aportes de los tres integrantes: Mauricio Cid en la lectura configurable y sus mediciones; Nayadeth Garrido en el núcleo POO de limpieza y validación y en los informes; y Víctor Bravo en los algoritmos, las mediciones y las visualizaciones. El archivo .mailmap consolida las identidades y git shortlog -sn muestra los aportes por integrante.

3.3 Eficiencia, validación y programación orientada a objetos

Se compararon cuatro implementaciones equivalentes mediante timeit: la referencia F2, una iterativa, una recursiva por división y conquista (Cormen et al., 2022) y una agrupada con pandas. Las pruebas usaron muestras de 100, 1.000, 10.000 y 44.226 filas, siete rondas de tres ejecuciones y orden alternado. En el conjunto completo, la variante agrupada alcanzó medianas de 19,02 ms por tamaño de proveedor y 20,96 ms por tipo de licitación, aproximadamente 1,16 veces más rápida que la referencia. El cambio más importante fue leer solo siete de 74 columnas: el flujo completo bajó de 1.199 a 506 ms y el DataFrame leído de 195,7 a 8,9 MiB. Estas cifras corresponden al equipo y a las versiones registradas, y no se pueden generalizar a otros entornos (Python Software Foundation, s. f.-a, s. f.-b; The pandas development team, s. f.-a, s. f.-b).

La variante recursiva divide el intervalo por el punto medio, resuelve directamente los bloques pequeños y combina los conteos parciales:

```
def _contar_dividiendo(grupos, codigos, inicio, fin, bloque_base):
    if fin - inicio <= bloque_base:          # caso base: conteo directo
        return _contar_rango(grupos, codigos, inicio, fin)
    medio = (inicio + fin) // 2
    izquierda = _contar_dividiendo(grupos, codigos, inicio, medio, bloque_base)
    derecha = _contar_dividiendo(grupos, codigos, medio, fin, bloque_base)
    for grupo, valores in derecha.items():   # combina los conteos parciales
        acumulado = izquierda.setdefault(grupo, [0, 0, 0])
        for i in range(3):
            acumulado[i] += valores[i]
    return izquierda
```

Todas las alternativas son lineales respecto del número de filas; las diferencias medidas son de constantes. La recursiva no ofreció una ventaja sostenida y conserva diccionarios parciales, por lo que se eligió la agrupada para el volumen completo.

Tabla 2*Arquitectura, principios orientados a objetos y verificación*

Componente	Decisión de diseño	Comprobación
Lectura	LectorDatos define el contrato y LectorCSV implementa la lectura.	Herencia y polimorfismo.
Limpieza	LimpiadorLicitaciones coordina transformaciones y protege su resumen interno.	La copia devuelta no altera el estado.
Validación	ReglaValidacion permite reglas intercambiables; el validador las compone.	Lista vacía rechazada; casos de error cubiertos.
Análisis	Cuatro funciones con los mismos parámetros, intercambiables entre sí.	Mismo resultado que F2 antes de medir.

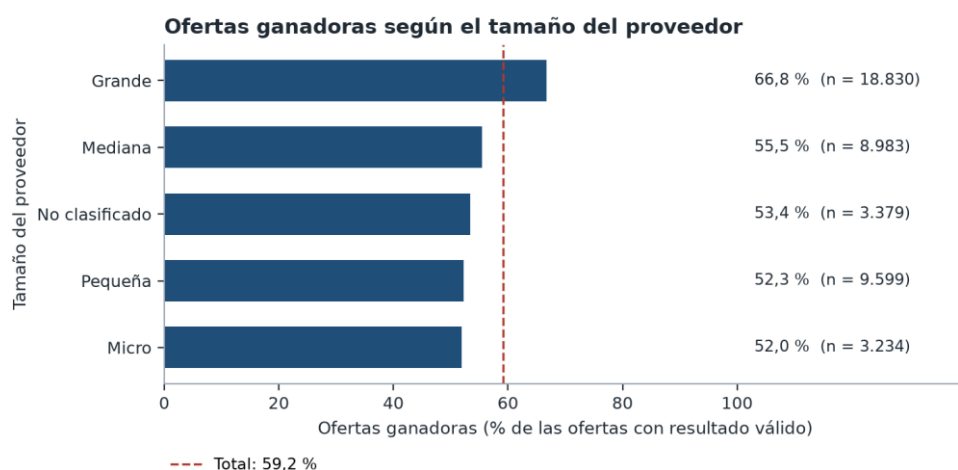
Nota. La suite de F3 contiene 29 pruebas y F4 añade 10 pruebas de visualización; las 39 finalizaron correctamente.

La herencia aparece en las interfaces abstractas LectorDatos y ReglaValidacion, definidas con el módulo abc (Python Software Foundation, s. f.-c). El polimorfismo permite que el validador invoque validar sin conocer la regla concreta, y el encapsulamiento impide que una modificación externa del resumen del limpiador cambie su estado. Separar lectura, limpieza y validación en clases con una sola responsabilidad sigue los criterios de diseño de Lott y Phillips (2021). El validador distingue None, que selecciona las reglas predeterminadas, de una lista vacía, que produce un error explícito.

4. Resultados y discusión

Las tablas de F4 se construyeron con 44.025 ofertas válidas pertenecientes a procesos adjudicados. Cada porcentaje utiliza su propio grupo como denominador. Las figuras se generan en F4/F4_Comunicacion.ipynb.

4.1 Resultados por tamaño de proveedor

Figura 1*Proporción de ofertas ganadoras según el tamaño del proveedor*

Procesos adjudicados; n = ofertas con resultado válido de cada grupo.
Fuente: elaboración propia con datos abiertos de ChileCompra, sector Salud, marzo de 2026.

Nota. Elaboración propia a partir de Datos Abiertos de ChileCompra, marzo de 2026. n = ofertas válidas

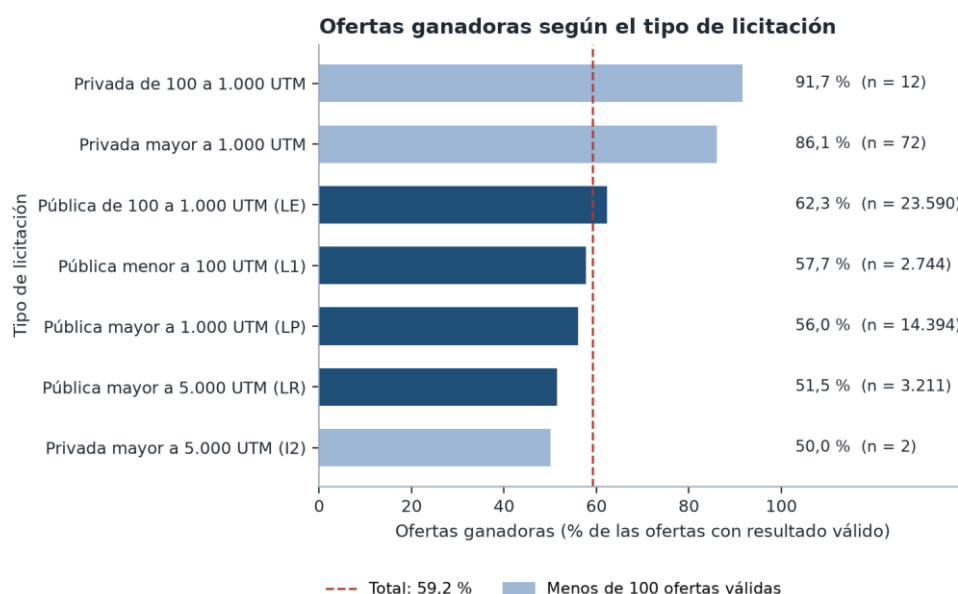
de cada grupo en procesos adjudicados.

Las empresas grandes ganan el 66,8 % de sus ofertas, 7,6 puntos porcentuales sobre el total y 14,8 puntos sobre las micro (52,0 %). El orden micro, pequeña, mediana y grande coincide con la hipótesis, pero el aumento no es gradual: micro, pequeña y mediana se concentran en un rango de 3,5 puntos y el salto se produce en las grandes. La evidencia respalda la dirección de la hipótesis, no una relación lineal entre tamaño y éxito. Además, las grandes reúnen 18.830 ofertas válidas, por lo que influyen de manera importante en el promedio general.

4.2 Resultados por tipo de licitación

Figura 2

Proporción de ofertas ganadoras según el tipo de licitación



Procesos adjudicados; n = ofertas con resultado válido de cada grupo.
Fuente: elaboración propia con datos abiertos de ChileCompra, sector Salud, marzo de 2026.

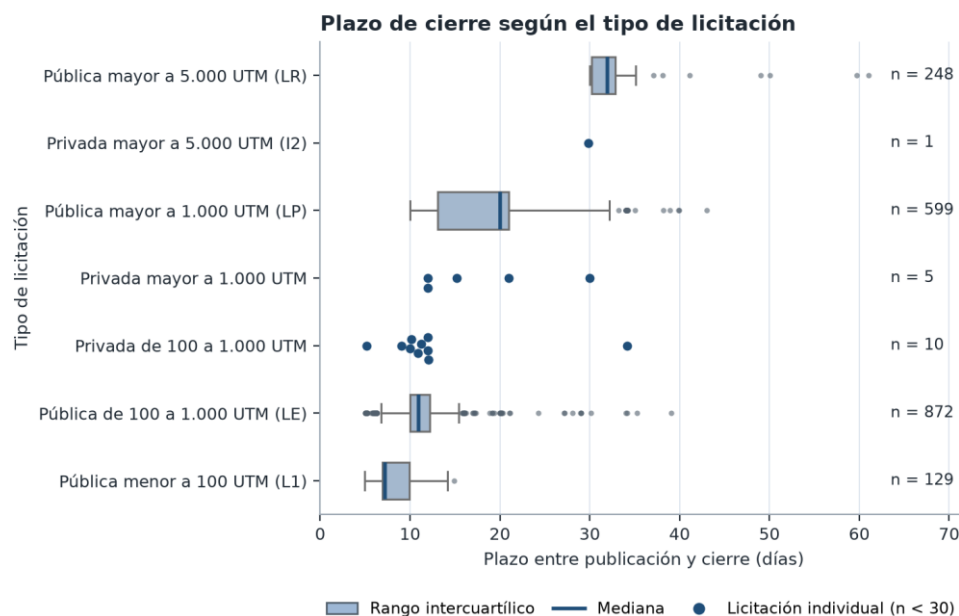
Nota. Elaboración propia a partir de Datos Abiertos de ChileCompra, marzo de 2026. Los tipos con menos de 100 ofertas se muestran en un tono más claro.

Las licitaciones públicas concentran el 99,8 % de las ofertas. En los procesos sobre 100 UTM, la proporción ganadora disminuye al aumentar el monto: 62,3 % en LE, 56,0 % en LP y 51,5 % en LR. Las licitaciones menores a 100 UTM alcanzan 57,7 % y no siguen ese orden. Los porcentajes de procesos privados, entre 50,0 % y 91,7 %, se basan en solo 2, 12 y 72 ofertas: se informan por transparencia, pero no permiten una comparación confiable.

4.3 Plazos de cierre y discusión

Figura 3

Distribución del plazo de cierre según el tipo de licitación



Una observación por licitación; n = número de licitaciones de cada tipo.
Fuente: elaboración propia con datos abiertos de ChileCompra, sector Salud, marzo de 2026.

Nota. Elaboración propia a partir de 1.864 licitaciones únicas. Los tipos con menos de 30 procesos se muestran como puntos individuales.

El plazo mediano de las licitaciones públicas aumenta con el monto: 7,2 días en L1, 11,0 en LE, 20,0 en LP y 32,0 en LR. Las cajas muestran un patrón escalonado y los plazos de hasta 61 días se concentran en procesos LR; por ello se conservaron: corresponden a procesos de mayor monto y no a errores de registro. Leído junto con la Figura 2, los procesos de mayor monto otorgan más tiempo para ofertar y presentan una menor proporción de ofertas ganadoras. Esta coincidencia es descriptiva y no demuestra que el plazo cause la diferencia.

Limitaciones y riesgos. El conjunto cubre un solo mes y el sector Salud: no permite observar tendencias temporales ni generalizar a todo el sistema. Los porcentajes por oferta dan mayor peso a licitaciones con más ítems u oferentes; 3.379 ofertas válidas no tienen tamaño de proveedor clasificado; y no se controlan monto ofertado, producto, institución compradora u otras variables. El principal riesgo es leer la diferencia entre tamaños como una ventaja causal. En términos técnicos, las mediciones dependen del equipo y de la caché, y tracemalloc no representa toda la memoria del proceso.

5. Conclusiones y recomendaciones

El proyecto responde la pregunta analítica con un flujo reproducible, modular y verificable. La hipótesis se cumple en su dirección: las empresas grandes presentan la mayor proporción de ofertas ganadoras; sin embargo, la diferencia se concentra en ese grupo y no forma una progresión uniforme. En las licitaciones públicas sobre 100 UTM, el porcentaje ganador disminuye al aumentar el monto, mientras el plazo mediano de cierre crece de 11 a 32 días.

La principal mejora técnica no provino solo de cambiar el algoritmo, sino de reducir los datos que ingresan al cálculo. Leer siete columnas con tipos definidos disminuyó el tiempo total y la memoria sin modificar los resultados. La variante agrupada resultó apropiada para el volumen completo, mientras que las alternativas iterativa y recursiva fueron competitivas en muestras pequeñas. Esta evidencia justifica la elección para este caso, no una regla general.

Como reflexión del equipo, separar lectura, limpieza y validación en clases facilitó repartir el trabajo y probar cada parte por separado, y medir antes de optimizar evitó decisiones basadas en intuición. La reducción de columnas descrita en la sección 3.3 muestra que definir con precisión qué datos requiere el análisis es tan importante como elegir el algoritmo. En términos de escalabilidad, todas las alternativas son lineales respecto del número de filas, por lo que el tiempo crece en proporción al volumen de datos; reducir las columnas disminuye el costo de cada fila procesada y permite ampliar el período o sumar sectores sin cambiar la arquitectura. En un entorno empresarial, esta eficiencia tiene un efecto directo en los costos: menos memoria y menos tiempo de cómputo significan una infraestructura más económica, sobre todo en servicios en la nube que cobran por uso. El beneficio aumenta cuando el análisis es recurrente, por ejemplo, si se repite con cada reporte mensual de ChileCompra, porque el ahorro de cada ejecución se acumula y el diseño modular permite automatizar el proceso sin rehacer el código. Como mejora futura se recomienda ampliar el período de observación, comparar sectores, analizar por licitación además de por oferta, incorporar variables de monto y producto, y repetir las mediciones en otros equipos.

6. Trazabilidad de mejoras

La Tabla 3 relaciona las observaciones recibidas en las evaluaciones anteriores con las acciones aplicadas. El archivo changelog.md del repositorio registra la fecha, la descripción, el commit y la justificación técnica de cada cambio.

Tabla 3

Observaciones recibidas y acciones incorporadas

Observación	Acción y commit	Impacto
La carpeta data/processed estaba vacía y el informe afirmaba que contenía el dataset.	El README declara que los derivados se regeneran con F2 e informa sus huellas SHA-256 (48e62d3).	Documentación coherente con el repositorio.
La portada indicaba MCDIA500 y no incluía el enlace al repositorio.	Código corregido y enlace en la portada (ab661e4).	Presentación formal correcta.
Una misma persona aparecía con varias identidades en Git.	Archivo .mailmap (87b1526, 9135329).	Autoría trazable por integrante.
Las funciones de F2 podían reorganizarse en clases.	Contrato, lector, limpiador y validador (8087c45, 189f251).	Modularidad y pruebas por componente.
Convenía medir la lectura con selección de columnas y tipos.	Lectura configurable y mediciones (7f66d24, 7a0ead5).	Flujo 2,4 veces más rápido y 95 % menos memoria.
Los notebooks mostraban pocos resultados intermedios.	Notebooks F3 y F4 con un resultado por celda (e41c1fd, 5da8f2a).	Resultados verificables paso a paso.

Nota. Los commits citados pueden consultarse en el historial del repositorio.

7. Bibliografía

- ChileCompra. (s. f.). *Datos abiertos del sistema de compras públicas*. <https://datos-abiertos.chilecompra.cl/descargas>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Lott, S. F., & Phillips, D. (2021). *Python object-oriented programming: Build robust and maintainable object-oriented Python applications and libraries* (4.^a ed.). Packt Publishing.
- McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media.
- Python Software Foundation. (s. f.-a). *timeit — Measure execution time of small code snippets*. Python 3 documentation. <https://docs.python.org/3/library/timeit.html>
- Python Software Foundation. (s. f.-b). *tracemalloc — Trace memory allocations*. Python 3 documentation. <https://docs.python.org/3/library/tracemalloc.html>
- Python Software Foundation. (s. f.-c). *abc — Abstract base classes*. Python 3 documentation. <https://docs.python.org/3/library/abc.html>
- Samuel, S., & Mietchen, D. (2024). Computational reproducibility of Jupyter notebooks from biomedical publications. *GigaScience*, 13, Artículo giad113. <https://doi.org/10.1093/gigascience/giad113>
- The pandas development team. (s. f.-a). *pandas.DataFrame.groupby*. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.groupby.html>
- The pandas development team. (s. f.-b). *pandas.read_csv*. pandas documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
- Universidad Andrés Bello. (2026a). *Fase 4: Presentación y comunicación del proyecto* [Material del curso MCDI500]. Canvas UNAB.
- Universidad Andrés Bello. (2026b). *Sumativa 3: Proyecto final integrador Fase 4 – Análisis, reproducibilidad y comunicación de resultados* [Pauta y rúbrica del curso MCDI500]. Canvas UNAB.
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>